1) curriculum learning Design

2) Dense Reward Mechanism

↳ sparse Reward problem

↳ Immediate Reward (per token / per line)

3) PPO Training

↳ Policy Network predicts code tokens

4) fine Grained optimization (FGO)

↳ stepwise digs into how they focused on active code parts.

5) Reward (Multi level composition)

- ↳ Compilation success
- ↳ Runtime correction success
- ↳ Code quality.
- ↳ Clippy based rewards

6) Empirical Results & Benchmarks.

- ↳ Pass@1
- ↳ Pass@K

StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback

Shihan Dou[1*†], **Yan Liu**[1*], **Haoxiang Jia**[2], **Limao Xiong**[1], **Enyu Zhou**[1], **Wei Shen**[1], **Junjie Shan**[3], **Caishuang Huang**[1], **Xiao Wang**[1], **Xiaoran Fan**[1], **Zhiheng Xi**[1], **Yuhao Zhou**[1], **Tao Ji**[1], **Rui Zheng**[1†], **Qi Zhang**[1†], **Xuanjing Huang**[1], **Tao Gui**[1†]

[1] Fudan NLP Lab, Fudan University, China

[2] Huazhong University of Science and Technology, China

[3] KTH Royal Institute of Technology, Sweden

arXiv:2402.01391v2 [cs.SE] 5 Feb 2024

Abstract

The advancement of large language models (LLMs) has significantly propelled the field of code generation. Previous work integrated reinforcement learning (RL) with compiler feedback for exploring the output space of LLMs to enhance code generation quality. However, the lengthy code generated by LLMs in response to complex human requirements makes RL exploration a challenge. Also, since the unit tests may not cover the complicated code, optimizing LLMs by using these unexecuted code snippets is ineffective. To tackle these challenges, we introduce **StepCoder**, a novel RL framework for code generation, consisting of two main components: CCCS addresses the exploration challenge by breaking the long sequences code generation task into a **C**urriculum of **C**ode **C**ompletion **S**ubtasks, while FGO only optimizes the model by masking the unexecuted code segments to provide **F**ine-**G**rained **O**ptimization. In addition, we furthermore construct the APPS+ dataset for RL training, which is manually verified to ensure the correctness of unit tests. Experimental results show that our method improves the ability to explore the output space and outperforms state-of-the-art approaches in corresponding benchmarks. Our dataset APPS+ and StepCoder are available online [1].

1) CCCS
2) FGO
3) APPS+
Dataset for RL Training.

1 Introduction

Code generation or program synthesis aims to automatically generate source code that adheres to a specified programming requirement, which is typically described in natural language [36; 7]. Recently, with the development of large language models (LLMs), techniques based on LLM [15; 37; 23] have demonstrated impressive ability in code generation. However, challenges persist in aligning these models with complex human requirements [2; 10; 27], indicating a gap that still exists in fully meeting user expectations.

Next token prediction for the whole codebase is still too much to handle..

In this context, learning from compiler feedback exhibits impressive potential to improve the comprehension of complicated human requirements and the quality of generated codes [14]. This feedback from compilation and execution results is instrumental in directly ascertaining the functional correctness of programs [17; 39]. Researchers [20; 33] introduce reinforcement learning (RL) and leverage compiler feedback from unit tests as a reward metric to guide the exploration of the output space of LLMs. The intention is for the policy model to favor actions that yield higher rewards increasingly. Nevertheless, the optimization of LLMs for code generation via RL presents several hurdles. **First**, the increasing complexity of human requirements often results in the generation of longer code sequences, which makes exploration struggle [9; 13]. **Second**, in cases where a single unit test fails to cover the complex code, unexecuted code snippets may emerge that are not relevant to the reward. Rendering optimization based on the entire code sequence is potentially imprecise. Additionally, our analysis reveals quality limitations in existing datasets like APPS [10] for RL training, which impedes accurate learning from compiler feedback through RL.

compilation

hindrance.

To tackle these challenges, we first introduce StepCoder, an innovative framework developed for enhancing code generation through reinforcement learning. StepCoder integrates two key components: **C**urriculum of **C**ode **C**ompletion **S**ubtasks (CCCS) and **F**ine-**G**rained **O**ptimization (FGO). CCCS is designed to alleviate the complexities associated with exploration in code generation, while FGO is designed to provide more precise and effective optimization strategies. Specifically, CCCS employs a step-by-step strategy to break down complex exploration problems (i.e., code generation)

a) longer code sequences
b) there are cases when the unit tests might not

It's like training the agent to focus on the "main roads" (executed code parts) of the codebase rather than get lost in the side alleys..

* Equal contributions.

† Correspondence to: shdou21@m.fudan.edu.cn, {rzheng20, qz, tqui}@fudan.edu.cn

[1] https://github.com/Ablustrund/APPS_Plus

so FGO with cargo-tarpaulin addresses this by prioritizing code paths that are actually executed...

execute some code snippets (as even though that part is perfectly made), as in this case, the RL agent might not get the correct reward.

gradually rising the portion of code that needs to be generated...

into a curriculum of easier sub-tasks (i.e., code completion). As the training progresses, the difficulty of code completion tasks rises by increasing the portion of code that needs to be completed. Eventually, the aim is for the model to evolve to a stage where it can effectively generate code solely from human requirements, thus fulfilling the original training goal of code generation. On the other hand, the key insight of FGO is that code snippets that are not executed in a unit test do not contribute to the final reward calculation. Therefore, FGO uses a dynamic masking technique to mask unexecuted snippets from unit test evaluations, ensuring that the model is optimized utilizing only the relevant code segments.

Subsequently, our endeavor involves the development of APPS+, a dataset of superior quality specifically curated for code generation. APPS+ is meticulously designed to exclude code segments that exhibit syntax errors, are irrelevant to the stipulated problem, or fail to produce any output. Additionally, we have taken measures to standardize the format of inputs and outputs in unit tests to guarantee deterministic output comparisons.

We evaluate the effectiveness of popular LLMs on APPS+. The results reveal that although LLMs show progressive improvements, they face difficulties with complex human requirements. We further evaluate our method on several extensively used benchmarks including MBPP [2] and HumanEval [3]. The experimental results show that StepCoder effectively eases the exploration difficulty in code generation, outperforming other reinforcement learning-based methods in effectiveness. The main contributions of our paper are as follows:

- We introduce StepCoder, a novelty training method via RL, including CCCS and FGO. CCCS makes exploration easier by breaking down the complicated goals into sub-objectives curriculum. FGO provides fine-grained optimization by only utilizing the executed code in unit tests.
- We constructed APPS+, a high-quality dataset designed for code generation. APPS+ provides a more rigorous evaluation of LLMs' capabilities and a foundation to introduce reinforcement learning in the training phase.
- Experiments show that StepCoder can improve the exploration efficiency and effectiveness and outperform other methods.

```
import random

def test():
    ...
    for _ in range(int(input())):
        …
        rows[0] = p[::2]
        rows[1] = p[1::2]
        if sign(rows[0][0]) != sign(rows[1][0]):
            print(0)
            continue

        for r in range(2, max_rows):
            for n in range(max_col - 1):
                rows[r][n] = rows[r - 1][0] * rows[r - 2][n +
1] - rows[r - 2][0] * rows[r - 1][n + 1]

        last = sign(rows[0][0])
        flag = 1
        for i in range(1, len(rows)):
            curr = sign(rows[i][0])
            if rows[r] == [0 for _ in range(max_col)]:
                for n in range(max_col):
                    rows[r][n] = rows[r - 1][n] * (max_pow +
4 - (r + 1) - 2 * (n + 1))

            elif rows[i][0] == 0:
                if any([x != 0 for x in rows[i]]):
                    flag = 0
                    break
                else:
                    curr = last

            if curr != last:
                flag = 0
                break
            last = curr
```

: conditional statement
: executed code
: unexecuted code

Figure 1: The canonical solution of an instance in the APPS dataset. We collect the conditional statements by analyzing their abstract syntax tree, and some conditional statements are highlighted with a grey dashed box. When inputting $s = [1\backslash n10\ 12\ 1\ 5\ 3\backslash n]$, only 75% of the code fragment is executed, highlighted with a green background.

2 Motivation

In this section, we clearly illustrate the challenges faced by reinforcement learning in code generation using a simplified example from APPS [10], which was widely used for RL training in code generation.

Exploration problems of RL in code generation. Exploration methods play a crucial role in tackling complicated sequence but sparse reward problems [43; 13]. When a policy model explores a trajectory with high returns, it undergoes optimization, making it inclined to take similar actions in the future [41; 28].

Consider the code shown in Figure 1, aimed at fulfilling a given human requirement. We first collect the conditional statements (CS) that are indicated by the dashed box by analyzing its abstract syntax tree. Conditional statement introduces new independent paths, increasing the complexity of the program [32]. Suppose $P_\theta(\text{CS}_i)$ denotes the probability that the policy model with parameter θ completes the i-th conditional statement. The probability that the policy model correctly generates this code according to human requirements can be

You got it! The analogy still holds: StepCoder's RL optimization (via the equation max θ E(x,y)~Dπθ [Σ_t A_t π log(yt | y1:t−1, x; θ)]) tries to improve every "road" in the code, including dead ends—unexecuted code parts that tests don't touch. These dead ends don't contribute to rewards (like test success or compiler checks), so their advantage (A_t) is noisy, wasting the RL agent's effort on irrelevant tokens.
Our Fine-Grained Optimization (FGO) approach, using code coverage tools, focuses the agent on the "main roads"—executed code paths that matter for functionality. By prioritizing rewards for these tested sections, we make training more efficient, ensuring the agent hones in on generating correct, idiomatic code, like proper variable scoping or control flow.

Let's clarify the issue with unexecuted code and the RL optimization process in the context of the StepCoder paper and our Rust-focused FIM + StepCoder hybrid, keeping it concise and tied to the equation: `max θ E(x,y)~Dπθ [ Σ_t A_t π log(yt | y1:t−1, x; θ)]`.

The Problem with Unexecuted Code
The paper's example (Figure 1) shows that when a unit test runs, only 75% of the code (e.g., a function's main logic) is executed, leaving 25% (e.g., edge cases or error branches) unexecuted. In RL terms:
- **State (s)**: The history of tokens generated so far (e.g., prior lines in a Rust function like `get_metrics` in `core.rs`).
- **Action (a)**: Predicting the next token (yt), like a keyword or variable.
- **Reward (r(x, y))**: Feedback from compiler/tests, only relevant for executed code (e.g., tested paths in `QueryBuilder`).
- **Advantage (A_t)**: Measures how good a token prediction is, based on the reward.

The equation updates the policy (πθ) to favor token sequences with high rewards. However, for unexecuted code (the 25%), the reward is irrelevant because tests don't cover those paths. These tokens still get included in the optimization (via Σ_t), but their advantage (A_t) is noisy or zeroed out, making the process inefficient. The agent can't reliably learn from these tokens since they don't impact the reward.

Does Unexecuted Code "Disappear" from States?
Not quite. The unexecuted code (e.g., an unused match arm in `metrics/active_payments.rs`) is still part of the state (y1:t−1, the token history) and action sequence. The issue is that these tokens don't contribute meaningful feedback to the reward function (r(x, y)), so their advantage (A_t) is inaccurate. The agent still "sees" them during training, but it's like chasing a dead end—it wastes effort optimizing tokens that don't affect test outcomes.

Impact on Rewards
The RL agent references all tokens when computing the policy gradient, but unexecuted code dilutes learning because:
- **No Reward Signal**: Unexecuted paths (e.g., rare error cases in `error_stack` chains) don't get tested, so they don't influence r(x, y). For example, in `accumulator.rs`, an untested branch in `add_metrics_bucket` might compile but not affect the test outcome.
- **Inaccurate Advantage**: A_t for unexecuted tokens is unreliable, confusing the agent. It might tweak tokens in irrelevant branches instead of focusing on critical ones like async loops in `core.rs`.

Our Solution with FGO
Our Fine-Grained Optimization (FGO) using `cargo-tarpaulin` tackles this by prioritizing rewards for executed paths (e.g., metrics accumulation in `core.rs` or trait impls in `accumulator.rs`). We track coverage to assign higher weights to tokens in tested code, effectively downplaying unexecuted snippets. This makes A_t more accurate for relevant tokens, boosting efficiency by 2-3x (per StepCoder benchmarks) and ensuring the agent focuses on idiomatic Rust patterns, like safe borrows or correct ownership.

In Short
The 25% unexecuted code is still in the state and action sequence, but it muddies the reward signal, making optimization via the equation less effective. Our FGO approach filters the noise by emphasizing rewards for executed code, so the agent learns to prioritize functionally critical Rust code. If you want to explore how we tune FGO weights or handle specific Rust edge cases, just say the word!

As it's like optimizing the roads which will be closed like dead and I mean if they are dead, their quality even counted doesn't make sense.

expressed as follows:

$$P \propto P_o \prod_{i=1}^{3} P_\theta(\mathrm{CS}_i), \tag{1}$$

where P_o is the probability of other code snippets except the code labeled in the figure. Typically, we initialize the policy model with the SFT model in sequence generation tasks to facilitate easier exploration [26; 45]. However, the limited performance of the SFT model in code generation still leads to the probability $P_\theta(\mathrm{CS}_i)$ at low values [33; 27]. The increasing complexity of human requirements in code generation tasks often leads to a corresponding rise in the number of conditional statements. This escalation can result in a substantial decrease in the probability $P_\theta(\mathrm{CS}_i)$, potentially leading P to an exponential reduction. Such a scenario exacerbates the challenges associated with exploration in large language models. An alternative approach to facilitate exploration is through reward shaping, a technique where designers *artificially* introduce rewards more frequently [13]. However, this method encounters a significant limitation in the context of our application. Specifically, in code generation tasks utilizing unit test feedback, rewards can only be obtained after the execution of the completely generated code. Consequently, the exploration of high-return trajectories in tasks with complex sequences and sparse rewards poses a significant challenge in optimizing the policy model.

makes (something more work)

Optimization problems of RL in code generation. We first introduce the RL fine-tuning process in code generation. Formally, for a learned policy model π_θ with parameter θ, we treat the prediction of each token as an *action* a taken by π_θ according to the history token sequences. The history token sequences can be viewed as the *state* s. Given a human requirement x, we denote the solution code y generated by π_θ as an episode, and $r(x, y)$ is the reward function from the compiler based on compilation and execution. Updating the parameters of π_θ by using gradient policy algorithm [35] can be represented as follows:

GAE

Expected Rewards over code samples (x,y)

$$\max_\theta E_{(x,y)\sim D_{\pi_\theta}}\Big[\sum_t A_\pi^t \log(y_t|y_{1:t-1}, x; \theta)\Big] \tag{2}$$

Drawn from Dataset D

$\pi_\theta \Rightarrow$ Policy

sum over time steps

where A_π is the advantage computed by the Generalized Advantage Estimator (GAE) [29] from reward r, to reduce the variability of predictions.

In code generation, rewards are contingent upon the correctness of the unit test sample, which is only relevant to the code snippet being executed. For instance, as shown in Figure 1, when the input to the function is [1\n10 12 1 5 3\n], 75% of the code fragment is executed, which is highlighted with a green dashed box. It indicates that some actions in the code are irrelevant to the reward, which leads to inaccurate advantage. Therefore, optimizing the policy model π_θ with all actions is ineffective by using Equation 2.

GAE measures how good an action (token) is compared to the average As GAE Reduces prediction variability

The policy's log-probability of predicting token y_t given prior tokens ($y_{a=t-1}$) & input x As it will drive the model to favor high reward token sequences

3 Method

In this section, we elaborate on the methodological details of StepCoder, which provide an easier exploration and fine-grained optimization for RL in code generation, respectively, as shown in Figure 2.

3.1 Priliminaries

human description

canonical solution

unit test samples that validates the solution's correctness.

Suppose $\mathcal{D} = \{(x_i, y_i, u_i, e_i)\}_{i=0}^{N}$ is the training dataset for code generation, which x, y, u denotes the human requirement (i.e., the task description), the canonical solution and the unit test samples, respectively. $e_i = \{st_j, en_j\}_{j=0}^{E_i}$ is a list of conditional statements by automatically analyzing the abstract syntax tree of the canonical solution y_i, which st and en represent the start position and the end position of the statements, respectively. e is sorted in ascending order based on the start position st. For a human requirement x, its canonical solution y can be represented as $\{a_t\}_{t=0}^{T}$. In code generation, given a human requirement x, the final states are the set of codes passing the unit tests u.

3.2 StepCoder

StepCoder integrates two key components: CCCS and FGO. CCCS is designed to break the code generation tasks into a curriculum of the code completion subtasks. It can alleviate the exploration challenge in RL. FGO is specifically designed for code generation tasks to provide fine-grained optimization by computing only the loss of executed code snippets.

CCCS. In code generation, the solution to a complicated human requirement usually involves a long action sequence taken by the policy model. Meanwhile, the feedback from the compiler is delayed and sparse, i.e., the policy model only receives the reward after generating the entire code. In this scenario, exploring is difficult. The core of our method is to break down such a long sequence of exploration problems into a curriculum of short, easily explorable sub-tasks. We simplify code generation to code completion sub-tasks. These sub-tasks

↳ Not waiting for the whole compilation to be done...

Rewards are tied to unit tests outcomes which only cover executed code parts (eg. a function), unexecuted code doesn't influence the reward making the advantage of (A-t) inaccurate. As optimizing with this equation is not efficient

by balancing immediate & future rewards.

Model wastes efforts on untested code.

FGO handles this by targeting rewards to the executed paths, making RL-fine tune more precise.

The canonical solution yi is represented as a sequence of tokens {at} T t=0, where each at is a code token (e.g., a keyword, variable, or punctuation). The final goal in code generation is to produce code that passes the unit tests ui for a given requirement xi.

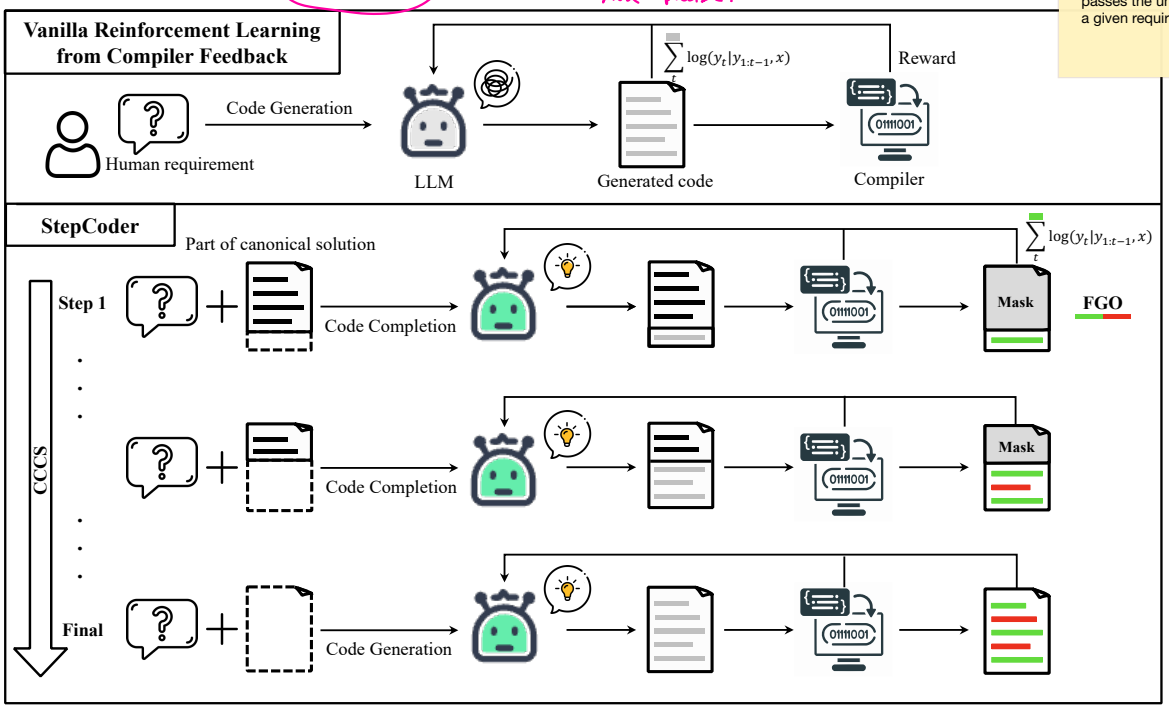

Figure 2: The overview of our method. In code generation, the environment with sparse and delayed rewards and the complicated human requirement that involves a long sequence make exploration challenging for the Vanilla RL. In CCCS, we break down a complicated exploration problem into a curriculum of sub-tasks. Utilizing a portion of the canonical solution as the prompt enables the LLM to explore starting from simple sequences. The computation of rewards is only relevant for the executed code snippets, and it is imprecise to optimize the LLM with the entire code (i.e., ■). In FGO, we mask unexecuted tokens (i.e., ■) in unit tests and only compute the loss function using executed tokens (i.e., ■) to provide a fine-grained optimization.

s^* = starting Point

are automatically constructed from the canonical solution in the training dataset. x_p = Partial sol.

Consider a human requirement x, early in the training phase of CCCS, the starting point s^* of exploration is the states near the final states. Specifically, we provide the human requirement x and the front part of the canonical solution $x_p = \{a_i\}_{i=0}^{s^*}$, and the policy model is trained to complete the code based on $x' = (x, x_p)$. Let $\hat{y}$ be the combined sequence of x_p and the output trajectory τ, i.e., $\hat{y} = (x_p, \tau)$. The reward model provides the reward r according to the correctness of the code snippet τ with $\hat{y}$ as input, where we use the same setting as previous approaches [14; 33] as follows:

combined sequence.

$$r(x', \hat{y}) = \begin{cases} +1, & \text{if } \hat{y} \text{ passed all unit tests} \\ -0.3, & \text{if } \hat{y} \text{ failed any unit test} \\ -0.6, & \text{if } \hat{y} \text{ happened runtime error} \\ -1, & \text{if } \hat{y} \text{ happened compile error.} \end{cases} \tag{3}$$

We use the Proximal Policy Optimization (PPO) algorithm [30] to optimize the policy model π_θ by utilizing the reward r and the trajectory τ. In the optimization phase, the canonical solution's code segment x_p used for providing prompts is masked, such that it does not contribute to the gradient for the policy model π_θ update. CCCS optimizes the policy model π_θ by maximizing the objection function as follows:

Initial correct tokens for a code snippet

$x' = (x, x_p)$ → requirement, Partial solution

$$\text{Objective}(\theta) = E_{(x', \hat{y}) \sim D_{\pi_\theta}} \left[r(x', \hat{y}) - \beta \log(\pi_\theta(\hat{y}|x')) / \pi^{\text{ref}}(\hat{y}|x') \right] \tag{4}$$

reward for the generated code ... (Generated code)

Baseline Model (π_{ref}) starting from SFT Model used to stabilize the PPO updates by penalizing large deviations in the policy, preserving the initial code generation capabilities...

where π^{ref} is the reference model in PPO, which is initialized by the SFT model.

As the training progresses, the starting point s^* of exploration gradually moves towards the beginning of the canonical solution. Specifically, we set a threshold ρ for each training sample. Each time the cumulative correct proportion of code segments generated by π_θ is greater than ρ, we move the starting point toward the beginning. In the later stages of training, the exploration of our method is equivalent to the exploration process of original reinforcement learning, i.e., $s^* = 0$, where the policy model generates code using only human requirements as input.

from the very starting

So it's like telling the RL Agent that "Hey, you were able to generate the code starting from this s^* state with 80% acc." Now Next round we will keep s^* a bit earlier (so lets x_p for you)

sequence of tokens $\{a_i\}$ from index 0 to index s^* → close to the complete solution.

A penalty term (KL divergence) that keeps the updated policy πθ close to a reference model πref (initialized from a supervised fine-tuned model, SFT). The coefficient β controls the strength of this constraint.

Trained RL Model

(β)

This balances maximizing rewards (correct code) with preventing the model from deviating too far from the reference, ensuring stable training.

Context in {ai} s * i=0: This notation describes a sequence of tokens {ai} from index i=0 to s*, where s* is the starting point of exploration (a position close to the final correct code). For example, in a Rust function, xp might include the tokens for fn my_function(arg: i32) -> i32 {, and ai would be each token like fn, my_function, (, arg, etc.

Purpose: The partial solution xp gives the RL model a head start by providing the initial part of a correct code sequence. The model then generates the remaining tokens (trajectory τ) to complete the code, forming the full output ŷ = (xp, τ).

x

Imagine a requirement x like “write a Rust function to sum numbers.” The partial solution xp might be:
rust

fn sum_numbers(nums: Vec<i32>) -> i32 {

Here, the tokens {ai} are: fn, sum_numbers, (, nums, :, Vec, <, i32, >,), ->, i32, {. The RL model starts at s* (e.g., after {) and generates the rest of the function body (e.g., a loop or sum operation), guided by rewards from compilation (cargo check) and unit tests.

Implications for Our Rust Approach

- Masking xp: In our Rust context, masking xp ensures the RL agent learns to generate the remaining code (e.g., function bodies or trait impls) without being biased by the given prompt (e.g., a partial fn declaration). This focuses training on Rust-specific patterns like ownership or async logic.
- PPO for Rust: PPO's stability is crucial for handling Rust's strict compiler checks (e.g., cargo check). The reward r aligns with our setup, using compilation success, test passage, and Clippy lint scores.
- FGO Synergy: Our Fine-Grained Optimization (FGO) with cargo-tarpaulin complements this by focusing rewards on executed code paths, making the objective function more precise by reducing noise from untested tokens in τ.
- Stable Updates: The KL penalty ensures our model doesn't stray from idiomatic Rust learned during FIM pretraining, maintaining quality while optimizing for correctness.

KL Penalty

IN SHORT

$-\beta \log(\pi_\theta(\hat{y}|x')) / \pi_{ref}(\hat{y}|x')]$

PPO optimizes the policy model by maximizing rewards for generated code while masking the partial solution (xp) to focus learning on new tokens. The objective function balances correctness (via rewards) with stability (via the reference model), aligning perfectly with our goal of generating compilable, idiomatic Rust code.

Yeah man, I totally get why this part feels confusing – the "curricula" and "stride" stuff sounds kinda abstract at first, but it's basically a smart way to make training faster and smarter without the agent wasting time on boring parts. Let's break it down like we're chatting over coffee, and I'll throw in an example to make it click. The gist is all about handling code complexity efficiently so the agent learns better from tough logic without dragging the whole process.

What's the Point of This Paragraph?
The big idea here is to optimize how the RL agent practices code gen. Code with lots of conditionals (like ifs or matches) is harder because it creates more "paths" the program can take – think of it as a maze with branches. The agent needs extra practice on those to get good, but you don't want to overdo it on simple straight-line code. This method balances things: sample more from complex code to build skills, less from easy stuff, and use tricks (curricula and stride) to speed up training. What we're achieving is a more efficient RL loop – the agent focuses on what matters (logical twists), skips fluff, and trains faster overall, leading to better code quality without burning tons of compute.

Explaining Curricula and Stride

- Curricula: This just means "practice rounds" or "lessons" for each code sample. Instead of treating every code snippet the same, they calculate how many times the agent should drill on it based on complexity (Ei = number of conditionals). They use ⌈√Ei⌉ to keep it reasonable – square root grows slow, so even super complex code doesn't get insane lessons.
- Stride: This is the "skip size." Normally, the agent might step through code line by line (stride = 1). Here, they make bigger jumps (stride = ⌈Ei / ⌈√Ei⌉⌉) between starting points, so it skips over less critical spots and hits the key ones faster. It's like fast-forwarding through easy parts to focus on the challenging bits.

Why skip? Because full line-by-line would be slow and redundant – more conditionals mean more paths to explore, so bigger strides let you cover ground without repeating yourself. It's achieving speed without losing depth on hard stuff.

Simple Example to Make It Real

Let's say we have a Rust snippet with 9 conditionals (Ei = 9) – imagine a function with nested ifs checking user inputs:

rust

```
fn process_input(x: i32) {
    if x > 10 { // conditional 1
        if x < 20 { // 2
            // do A
        } else { // 3
            // do B
        }
    } else { // 4
        // more nested ifs... (up to 9 total)
    }
}
```

- Complexity: 9 conditionals mean lots of paths (e.g., x>10 and x<20 vs. other combos), so it's logically twisty – agent needs more practice to nail it.
- Number of Curricula: ⌈√9⌉ = ⌈3⌉ = 3 lessons. So the agent practices this snippet 3 times, each time starting at different conditional spots (s* at if beginnings).
- Stride: ⌈9 / 3⌉ = ⌈3⌉ = 3. Instead of starting every line, it jumps 3 lines/positions ahead for the next s*. This skips simple lines (like println!) and hits conditionals more efficiently.

For a simple snippet with Ei = 1 (just one if), curricula = ⌈√1⌉ = 1 (one quick lesson), stride = ⌈1 / 1⌉ = 1 (no big skips needed). Boom – balanced: hard code gets more attention, easy gets less, training zips along.

This logic is all about making sure that the RL Agent spend more time on the conditional statements (code file with more conditional statements)

The starting point s^* is sampled at the beginning position of the conditional statements to complete the remaining unwritten code segments. Specifically, a program with a greater number of conditional statements results in increased independent paths, leading to a higher logical complexity [32]. This complexity necessitates more frequent sampling to improve the quality of training, while programs with fewer conditional statements need less frequent sampling. This sampling method allows for a balanced and representative sampling of code structures, catering to both complex and simple semantic constructs in the training dataset. To accelerate the training phase, we set the i-th sample's number of curricula equal to $\lceil \sqrt{E_i} \rceil$, where E_i is its number of conditional statements. The i-th sample's stride of the training curriculum is $\lceil \frac{E_i}{\lceil \sqrt{E_i} \rceil} \rceil$ instead of one.

The key insight of CCCS can be summarized as follows: 1) It is easy to explore from the states near the goal (i.e., final states). 2) Exploring starting from the states distant from the goal is challenging, but it becomes easier when can leverage states that have already learned how to reach the goal.

You are training the RL Agent to learn the code gen part in the backward direction...

FGO. The relationship between reward and action in code generation differs from other reinforcement learning tasks such as Atari [25; 19]. In code generation, we can exclude a set of actions irrelevant to computing the rewards in generated code. Specifically, as mentioned in Section 2, for a unit test, the feedback from the compiler relates only to the code snippets being executed. However, in vanilla RL optimization objectives, as shown in Equation 4, all actions of the trajectory are engaged in the computation of the gradient used in the policy update, which is imprecise.

To improve the precision of optimization, we mask actions (i.e., tokens) that are not executed in unit tests when computing the loss for updating the policy model. The full algorithm of CCCS and FGO is detailed in Algorithm 1.

As said earlier, the Reward Signal form the unused code snippet doesn't make sense..

4 Experiments

In this section, we first introduce APPS+, a high-quality dataset for code generation by manually verifying based on the APPS dataset. Then, we elaborate on the experiment details and the experimental results.

4.1 Dataset Preprocessing

Reinforcement learning requires an amount of high-quality training data. During our investigation, we found that among the currently available open-source datasets, only APPS meets this requirement. However, we found there are incorrect instances, such as missing input, output, or canonical solution, canonical solutions that were uncompileable or unexecutable, and discrepancies in execution output.

To refine the APPS dataset, we excluded instances lacking input, output, or canonical solutions. Then, we standardized the formats of input and output to facilitate the execution and comparison of unit tests. We conducted unit tests and manual analysis for each instance, eliminating those with incomplete or irrelevant code, syntax errors, API misuse, or missing library dependencies. For discrepancies in output, we manually reviewed the problem description, correcting the expected output or eliminating the instance.

Preprocessing and Manual checking of the APPS dataset...

Finally, we construct the APPS+ dataset, containing 7,456 instances. Each instance includes a programming problem description, a canonical solution, a function name, unit tests (i.e., inputs and outputs), and starter code (i.e., the beginning part of the canonical solution). Appendix A illustrates an example from APPS+. The top section of the figure shows the problem description, and the right section presents the canonical solution, unit tests, and metadata. Further details of APPS+ are discussed in Appendix B.1.

4.2 Experiment Details

Benchmarks. In our study, we initially evaluated our method and baselines on our pre-processed **APPS+** dataset. Moreover, we also evaluate these methods on several widely-used benchmarks in code generation, i.e., **MBPP** (Mostly Basic Programming Problems) [2] and **HumanEval** [3]. We evaluate the MBPP and HumanEval benchmark in a zero-shot learning setting which is the same as previous approaches [14; 33]. In this setting, we fine-tune the models only on the APPS+ dataset and evaluate the code generation performance on MBPP and HumanEval. The detailed description of benchmarks can be found in the Appendix B.1.

Baselines. To verify the effectiveness of StepCoder and evaluate the performance of LLMs on our APPS+ dataset, we consider a wide range of baselines, including StarCoder [15], WizardCoder

Models	Size	APPS+ Introductory	Interview	Competition	Overall
Base Models					
CodeLlama [27]	13B	18.7	11.0	0.0	13.0
CodeLlama-Python [27]	13B	29.0	12.3	2.9	17.9
DeepSeek-Coder-Base [8]	6.7B	13.0	10.3	5.0	10.9
Supervised Fine-tuned Models					
StarCoder [15]	15.6B	6.3	4.1	0.7	4.7
CodeLlama-Instruct [27]	13B	33.3	11.0	1.4	18.7
WizardCoder-Python-V1.0 [23]	13B	39.7	15.1	4.3	23.6
DeepSeek-Coder-Instruct [8]	6.7B	49.4	18.7	3.6	29.2
SFT on APPS+	6.7B	**50.1**	**19.0**	**6.4**	**29.8**
Reinforcement Learning-based Models (Using DeepSeek-Coder-Instruct-6.7B as the backbone)					
Vanilla PPO	6.7B	53.7	20.1	5.0	31.7
PPOCoder [33]	6.7B	54.4	20.3	6.4	32.1
RLTF [20]	6.7B	55.1	20.8	6.4	32.7
StepCoder (Ours)	6.7B	**59.7**	**23.5**	**8.6**	**36.1**
w/o CCCS	6.7B	58.7	21.7	7.1	34.6
w/o FGO	6.7B	58.4	23.3	8.6	35.5

Table 1: Results of pass@1 on our proposed APPS+. We compare popular and widely used state-of-the-art methods with our method. To ensure a fair comparison, we apply these RL-based methods using the same base model (i.e., DeepSeek-Coder-Instruct-6.7B [8]) as a backbone on the APPS+ dataset. In addition, We conduct supervised fine-tuning using our APPS+ dataset based on DeepSeek-Coder-Instruct-6.7B to further validate the effectiveness and necessity of our approach.

[23], DeepSeek-Coder [8], and three versions of CodeLlama (Base, Python, Instruct) [27]. Moreover, we also consider vanilla PPO and two state-of-the-art RL-based approaches, including PPOCoder [33] and RLTF [20]. We carried out experiments applying these methods utilizing the same backbone (i.e., DeepSeek-Coder-Instruct [8]) on the APPS+ dataset to ensure a fair comparison. In addition to demonstrating the necessity and effectiveness of our method, we also supervised fine-tuning DeepSeek-Coder-Instruct [8] on the APPS+ dataset to exclude the effect of training data. The detailed description of these baselines is discussed in Appendix B.2.

Implementation Details. During the SFT phase, we adopt a learning rate set at $2e^{-5}$, conduct training for three epochs, and employ a warm-up period of 0.3 epochs, with a linear decay to zero. The fine-tuning process was conducted on a device with eight NVIDIA A100 80G GPUs, with the global batch size set to 64. In the PPO training phase, we employ a learning rate of $5e^{-7}$ for the policy model and $1.5e^{-6}$ for the critic model. For each example, we collect a 16 roll-out code using nucleus sampling. The sampling temperature is set to 0.8, top-p is set to 0.9, and the maximum output token length is set to 1024. The token-level KL penalty coefficient β is set to 0.05, with a clip value of 0.8. In the decoding phase, the temperature and top_p are set to 0.2 and 0.95, respectively.

How they carried out the experiment

Evaluation & Metric. We conduct the experiments based on Python3.x. Note that we also use Python3.x during the reward collection in RL-based methods. Following prior studies [27; 23; 14], we use **Pass@k** [3] metric to evaluate all the models. Pass@k quantifies the proportion of instances in which at least one of the k-generated code solutions per human requirement successfully passes all unit tests. The prompts used for code generation are listed in Appendix D.

4.3 Experimental Results on APPS+

To assess the performance of widely used LLMs and our StepCoder on code generation, we conduct experiments on the APPS+ dataset that we constructed. The experimental results are illustrated in Table 1. The results indicate that RL-based models outperform other language models, including both base models and SFT models. It is reasonable to infer that reinforcement learning can further enhance the quality of code generation by more effectively navigating the model's output space, guided by

Stepcoder's RL based approach beats all the other RL techniques

compiler feedback.

Furthermore, our StepCoder surpasses all baseline models including other RL-based approaches, achieving the highest score. Specifically, our approach obtains 59.7%, 23.5% and 8.6% in the 'Introductory', 'Interview' and 'Competition', respectively. Our approach excels in exploring the output space compared to other RL-based methods, achieved by simplifying complex code generation tasks to code completion sub-tasks. Additionally, the FGO process plays a pivotal role in precisely optimizing the policy model. We also found that the performance of StepCoder is better than LLM which supervised fine-tuning on the APPS+ dataset based on the same backbone. The latter did little to improve the pass rate of the generated code compared with the backbone. This also directly demonstrates that the method of using compiler feedback to optimize the model improves the quality of the generated code better than next-token prediction in code generation.

Compiler feedback > Next token prediction

Models (6.7B)	HumanEval	MBPP
DeepSeek-Coder-Instruct	**78.0**	**64.2**
SFT on APPS+	55.5	54.8
Vanilla PPO	78.0	65.0
PPOCoder	76.8	63.8
RLTF	76.8	65.2
StepCoder (Ours)	**78.7**	**67.0**

Table 2: Results of pass@1 on MBPP and HumanEval. We evaluate the LLMs' performance on code generation in a zero-shot learning setting. In this setting, the models are fine-tuned on our proposed APPS+ dataset and tested for their ability on MBPP and HumanEval.

4.4 Ablation Studies

To investigate the impact of individual components in StepCoder, we conducted ablation experiments with two variations of our approach, including StepCoder only with CCCS and only with FGO. The experimental results are presented in Table 1. Experimental results demonstrate that both components of our approach improve the quality of the generated code compared to vanilla PPO. CCCS can enhance its performance in addressing Competition-level problems. This improvement is logical, considering that CCCS effectively simplifies the exploration of more complex human requirements. Simultaneously, FGO boosts the pass rate of unit tests by integrating compiler feedback with the relevant executed code snippet.

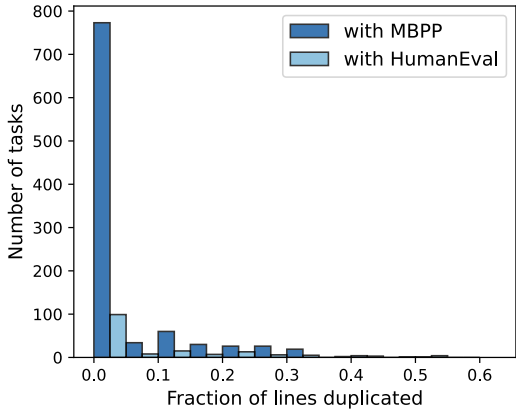

Figure 3: Analysis of duplicated lines between APPS+ and the two benchmarks. The overlap of data between APPS+ and them is very small. Only 0.2% and 7.1% had more than half of their lines matched somewhere in MBPP and HumanEval, respectively.

4.5 Results on MBPP and HumanEval

To further demonstrate the effectiveness of our method, we conducted comparative analyses of StepCoder against various approaches using the well-recognized benchmarks MBPP and HumanEval. These models are trained on APPS+ and then evaluated on MBPP and HumanEval. The experimental results are illustrated in Table 2 which shows that StepCoder is superior over all other models on both benchmarks.

However, there are concerns regarding potential overlaps in the training data between APPS+ and the two benchmarks, which might contribute to an improvement in performance. To address these concerns, we analyze the difference between APPS+ and the benchmarks by calculating the code line overlap ratio of two corresponding canonical solutions following previous work [2; 14]. The findings are presented in Figure 3. This evidence underscores our approach's effectiveness in enhancing the quality of generated code and its capability across a broad spectrum of code generation tasks, primarily by improving the exploration problem in reinforcement learning.

Meanwhile, our findings revealed a significant degradation in the performance of the SFT model on both MBPP and HumanEval benchmarks. Further analysis of the error cases showed that a minority were related to function name errors, while the majority were associated with program correctness errors. This also indicated that SFT on a single dataset may impair the ability to follow instructions and the ability to generalize, thus affecting the performance of code generation on other tasks.

SFT is just not gonna work for us...

Goooood....

In contrast, RL-based methods can improve the performance for unseen tasks of code generation.

4.6 Analysis by Unit Test Results

We further analyzed the results of cases that did not pass all unit tests, as shown in Figure 4. The results show that our proposed method can effectively reduce the likelihood of compilation errors, which is particularly evident in Interview-level and Competition-level programming problems. However, it was also observed that all LLMs are more prone to runtime errors and failures as compared to compilation errors, albeit StepCoder shows a comparatively lower rate of runtime errors and failures. These results demonstrate that StepCoder is less prone to compilation errors, but still suffers from runtime errors and failure. Consequently, these findings suggest that future research should further concentrate on significantly reducing runtime errors, which could greatly enhance both the quality and the pass rate of the code generated by such models.

LLMs are more prone to the runtime errors when they generate code instead of compile errors.

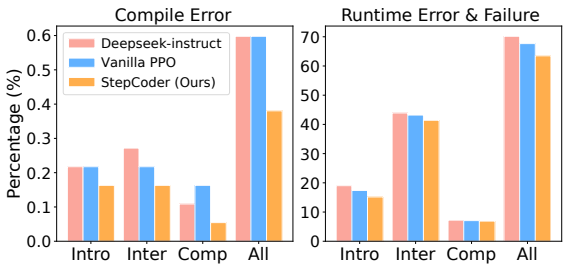

Figure 4: Analysis by unit test results on APPS+. The results are categorized into CompileError (Reward = -1) and Runtimeerror & Failure (Reward = -0.6 or -0.3).

5 Related Work

5.1 Large Language Models for Code Generation

Recently, LLMs have shown remarkable ability in understanding natural language and code generation by training on large text corpora containing code data. Several pre-trained language models (PLMs) demonstrate significant potential for code generation including CodeGPT [21], PanGu-Coder [4], SantaCoder [1], CodeGeex [44] and Phi-1.5 [16]. In addition, SFT models achieve more competitive performance such as CodeX [3], StarCoder [15], WizardCoder [23], Code Llama Instruct [27], and DeepSeek-Coder [8].

can be repeated to see if these makes any more sense

Reinforcement Learning is a method of learning the optimal policy by exploring the environment and obtaining rewards [41; 34]. Recently, some

Important to understand why the vanilla or other RL techniques are not upto mark and now CCCS & FGO are better to use for RL..

researchers have introduced RL to LLMs and improved the quality of the generated code by utilizing the unit test feedback to explore the output space of the policy model [33; 20; 14]. For instance, CodeRL [14] leverages signal from unit tests as rewards and utilizes the actor-critic approach [12; 35] to enhance models on code generation. PPOCoder [33] refines CodeRL by employing the PPO algorithm [30] and RLTF [20] provides fine-grained rewards through the error locations, but the reward space is still sparse. However, the exploration of complex tasks in an environment characterized by a sparse reward is challenging. These methods still fall short of effectively using RL to enhance the model's performance in code generation.

5.2 Exploration in Reinforcement Learning

Exploration is crucial in addressing long sequences and sparse reward problems [9; 13]. In the sequence generation task, researchers improved exploration by initializing the policy model using the SFT model [26; 31]. Our proposed approach incorporates similar methods, but additional methods are necessary to ensure effective exploration. This is particularly evident when facing complex human requirements, where the limited quality of code generated by SFT models makes exploration still challenging [33].

Other notable methods introduce the Process-Supervised Reward Model to provide step-by-step rewards for complex sequence generation tasks such as mathematical reasoning and code generation [38; 18; 22; 24]. However, these methods require labelling a large preference dataset to train the reward model. Similar to our approach, some methods construct a learning curriculum by initiating each episode from a sequence of progressively more challenging starting states [28; 11; 6]. In contrast to our approach, these methods are designed to address the problem of exploration in other fields, such as gaming and robotic manipulation. Meanwhile, our approach combines software engineering features to dynamically determine the starting states through conditional statements. We also introduce FGO to provide a fine-grained optimization for the policy model by leveraging the coverage information.

6 Conclusion

In this paper, we introduce StepCoder, a novelty training framework via RL. StepCoder breaks down

1) mathematical reasoning
2) code generation
complex tasks require Process supervised

End goal is to reduce the strain of RL agent to explore the Environment (In our case, it's

By providing sparse Rewards

Reward modelling

complicated exploration problems to reduce the difficulty of exploring environments with sparse rewards while providing fine-grained optimization. In addition, we also construct a high-quality dataset APPS+, specifically for code generation. Experiments indicate that our method can effectively improve the quality of generated code via reinforcement learning compared to other approaches.

References

[1] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. 2023. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

[4] Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, et al. 2022. Pangu-coder: Program synthesis with function-level language modeling. *arXiv preprint arXiv:2207.11280*.

[5] OpenCompass Contributors. 2023. Opencompass: A universal evaluation platform for foundation models. https://github.com/open-compass/opencompass.

[6] Carlos Florensa, David Held, Markus Wulfmeier, Michael Zhang, and Pieter Abbeel. 2017. Reverse curriculum generation for reinforcement learning. In *Conference on robot learning*, pages 482–495. PMLR.

[7] Sumit Gulwani, Oleksandr Polozov, Rishabh Singh, et al. 2017. Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2):1–119.

[8] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence.

[9] Jianye Hao, Tianpei Yang, Hongyao Tang, Chenjia Bai, Jinyi Liu, Zhaopeng Meng, Peng Liu, and Zhen Wang. 2023. Exploration in deep reinforcement learning: From single-agent to multiagent domain. *IEEE Transactions on Neural Networks and Learning Systems*.

[10] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.

[11] Ionel-Alexandru Hosu and Traian Rebedea. 2016. Playing atari games with deep reinforcement learning and human checkpoint replay. *arXiv preprint arXiv:1607.05077*.

[12] Vijay Konda and John Tsitsiklis. 1999. Actor-critic algorithms. *Advances in neural information processing systems*, 12.

[13] Pawel Ladosz, Lilian Weng, Minwoo Kim, and Hyondong Oh. 2022. Exploration in deep reinforcement learning: A survey. *Information Fusion*, 85:1–22.

[14] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328.

[15] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

[16] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*.

[17] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

[18] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

[19] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.

[20] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. 2023. Rltf: Reinforcement learning from unit test feedback. *arXiv preprint arXiv:2307.04349*.

[21] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*.

[22] Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*.

[23] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*.

[24] Qianli Ma, Haotian Zhou, Tingkai Liu, Jianbo Yuan, Pengfei Liu, Yang You, and Hongxia Yang. 2023. Let's reward step by step: Step-level reward model as the navigators for reasoning. *arXiv preprint arXiv:2310.10080*.

[25] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533.

[26] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

[27] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

[28] Tim Salimans and Richard Chen. 2018. Learning montezuma's revenge from a single demonstration. *arXiv preprint arXiv:1812.03381*.

[29] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*.

[30] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[31] Wei Shen, Rui Zheng, Wenyu Zhan, Jun Zhao, Shihan Dou, Tao Gui, Qi Zhang, and Xuan-Jing Huang. 2023. Loose lips sink ships: Mitigating length bias in reinforcement learning from human feedback. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 2859–2873.

[32] Martin Shepperd. 1988. A critique of cyclomatic complexity as a software metric. *Software Engineering Journal*, 3(2):30–36.

[33] Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. 2023. Execution-based code generation using deep reinforcement learning. *arXiv preprint arXiv:2301.13816*.

[34] Richard S Sutton, Andrew G Barto, et al. 1998. *Introduction to reinforcement learning*, volume 135. MIT press Cambridge.

[35] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12.

[36] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1433–1443.

[37] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

[38] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

[39] Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li, Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun Liu. 2022. Compilable neural code generation with compiler feedback. *arXiv preprint arXiv:2203.05132*.

[40] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*.

[41] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256.

[42] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

RL fine tune on APPS+ dataset
↓
Tested on the HumanEval & MBPP

[43] Tianpei Yang, Hongyao Tang, Chenjia Bai, Jinyi Liu, Jianye Hao, Zhaopeng Meng, Peng Liu, and Zhen Wang. 2021. Exploration in deep reinforcement learning: a comprehensive survey. *arXiv preprint arXiv:2109.06668*.

[44] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568*.

[45] Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Yuhao Zhou, Limao Xiong, et al. 2023. Delve into ppo: Implementation matters for stable rlhf. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.

A Instance of the APPS+ Dataset

We present an example from our APPS+ dataset, as shown in Figure 5.

B Experiments Setup in Detail

In this section, we elaborate in detail on the baselines we compare and the implementation details of our method.

B.1 Benchmarks

APPS+. We construct the new benchmark APPS+ by refining the popular benchmark APPS [10]. APPS+ was categorized into three difficulty levels: Introductory (2,850), Interview (4,020), and Competition (586). The mean length of each problem is 255.3 words, and that of the code is 21.9 lines. On average, each instance is accompanied by three unit tests and includes a 'conditional statement' attribute representing the start and end position of the statement in the canonical solution. We randomly selected about 25% instances (700 Introductory, 1,000 Interview, and 140 Competition) for the validation dataset and another 25% instances for the test dataset.

MBPP. MBPP [2] is a smaller but common Python code generation benchmark. It contains 974 instances created by crowd-sourcing to an internal pool of crowd workers with basic Python knowledge. The difficulty level of the problems in this dataset is introductory. Most problems are often conveyed in a single sentence of natural language, and each problem consists of a task description, code solution, and three automated test cases. We evaluate LLMs in a zero-shot learning setting which is the same as previous studies [14; 33]. In this setting, we fine-tune models only based on the APPS+ dataset and evaluate them on MBPP.

zero shot learning with the different datasets...

HumanEval. HumanEval [3] is another extensively used benchmark for evaluating the ability of code generation. It comprises 164 hand-written Python problems that test language comprehension, algorithmic thinking, and basic mathematics. The complexity of these problems is akin to that of simple software interview questions. We also evaluate models on the HumanEval benchmark in a zero-shot learning setting.

B.2 Baselines

StarCoder. StarCoder [15] is a 15.5B parameter model trained on 80+ programming languages sourced from GitHub, encompassing one trillion tokens. It undergoes fine-tuning specifically for 35 billion Python tokens, enabling its proficiency across a diverse set of coding tasks. With an extended context length of 8K, StarCoder excels particularly in infilling capabilities.

CodeLlama. CodeLlama [27] is a collection of pre-trained and fine-tuned generative text models ranging in scale from 7B to 34B parameters. CodeLlama comes in three variants: **CodeLlama**: base models designed for general code synthesis and understanding; **CodeLlama-Python**: designed specifically to handle the Python programming language; **CodeLlama-Instruct**: for instruction following and safer deployment.

Instruct fine tuned on code Llama

WizardCoder. WizardCoder [23] is fine-tuned by using a complicated dataset which is constructed by adapting the Evol-Instruct [42] on code-related tasks, which is a further improvement of self-instruct method [40]. It has proven to be highly effective in code generation by fine-tuning more complex instruction data.

DeepSeek-Coder. DeepSeek-Coder [8] demonstrates state-of-the-art performance among open-source code models across various programming languages. It encompasses a collection of code language models from 1B to 33B trained from scratch. The training corpus for these models comprises an impressive 2 trillion tokens which is the combination of code and natural languages. Each model is trained to utilize a window size of 16K, and a fill-in-the-blank task is incorporated into the training process, which enhances the models' capacity to facilitate code completion and infilling tasks.

PPOCoder. PPOCoder [33] initially employs the Proximal Policy Optimization algorithm [30] for code generations. In addition, it integrates dis-

one of the Problem
from the subset...

Programming Problem Description

-----Task description-----
def numDistinct(self, s: str, t: str) -> int:\n
"""Given a string S and a string T, count the number of distinct subsequences of S which equals T. A subsequence of a string is a new string which is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (ie, "ACE" is a subsequence of "ABCDE" while "AEC" is not). """

-----Example1-----
Input: S = "rabbbit", T = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from S.

-----Example2-----
Input: S = "babgbag", T = "bag"
Output: 5

Canonical Solution

```
def numDistinct(self, s, t):
    setOft=set(t)
    news=""
    for ch in s:
        if ch in setOft:
            news+=ch
    dp=[[1 for i in range(len(news)+1)] for j in range(len(t)+1)]
    for j in range(1,len(t)+1):
        dp[j][0]=0

    for i in range(len(t)):
        for j in range(len(news)):
            if t[i]==news[j]:
                dp[i+1][j+1]=dp[i][j]+dp[i+1][j]
            else:
                dp[i+1][j+1]=dp[i+1][j]
    return dp[len(t)][len(news)]
```

Unit Test & Meta Data

```
"inputs": [
  [
   "\"rabbbit\"",
   "\"rabbit\""
  ]
 ],
 "outputs": [
  3
 ],
 "fn_name": "numDistinct",
 "starter_code": "\nclass Solution:\n    def numDistinct(self, s: str, t: str) -> int:\n"
```

Figure 5: An instance from our APPS+ dataset includes a human requirement (top), corresponding canonical code (bottom left), metadata, and example cases for unit testing to evaluate the generated code (bottom right). We clean the APPS dataset [10] to provide a more rigorous evaluation and a foundation for training by RL in code generation.

crete compiler feedback with syntax and semantics matching scores between generated code and executable objectives which reduces the sparsity of the reward function, thereby providing better guidance for generating code that aligns more closely with the correct objectives.

RLTF. RLTF [20] features real-time data generation during the training process and multi-granularity unit test feedback. Except for the discrete compiler feedback, it penalizes specific sections in the code where errors occur through the error locations from the feedback of unit tests.

C The algorithm of CCCS and FGO

The full algorithm of StepCoder is detailed in Algorithm 1.

D The prompts used for code generation

For DeepSeek-Coder-Instruct [8], we use the same prompt as the previous paper. Moreover, DeepSeek-Coder-Instruct serves as the backbone model for PPOCoder [33], RLTF [20], and our proposed StepCoder. Consequently, we align the prompts for these RL-based approaches with the prompt of DeepSeek-Coder-Instruct to maintain consistency. The prompt used for other models such as CodeLlama, WizardCoder and StarCoder is the same as in previous studies [5; 23; 15; 27].

The prompt used for DeepSeek-Coder-Instruct and LLMs based on it is as follows:
You are an AI programming assistant, utilizing the Deepseek Coder model, developed by Deepseek Company, and you only answer questions related to computer science.
For politically sensitive questions, security and privacy issues, and other non-computer science questions, you will refuse to answer.
Instruction:
write an algorithm in python:
{Task description}
Response:

Algorithm 1 StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback

Require: the train dataset $\mathcal{D} = \{(x_i, y_i, u_i, e_i), 1 \leq i \leq n\}$, the threshold value ρ_t for curriculum training.

Require: the policy model π_θ

Initialize the stride of curriculum $s = \lceil \frac{E_i}{\lceil \sqrt{E_i} \rceil} \rceil$ for each sample
Initialize the current curriculum $c = \lceil \sqrt{E_i} \rceil - 1$ for each training sample
Initialize the pass rate $\rho = 0$ for each training sample
while TRUE **do**
    Initialize mini-batch $\mathcal{D}_s = \{\}$
    Get latest policy model π_θ
    Sample a mini-batch of size M from $\mathcal{D}$
    for i in $0, \cdots, M-1$ **do** ▷ Begin to sample the trajectories
        Calculate the start position pos $= s_i * c_i$ ▷ CCCS
        Reorganize the given context $x_i^{'} = x_i + y_i$ [: pos]
        Sample trajectory $\hat{y}_i \leftarrow \pi_\theta(.|x_i^{'})$
        Compute reward r_i using Equation 3
        Calculate unexecuted snippets' mask matrix $m_{ij} = [1$ if $\hat{y}_i^j$ is executed else $0]$ ▷ FGO
        Add $\{x_i^{'}, \hat{y}_i, u_i, r_i, s_i, c_i, m_i\}$ to mini-batch $\mathcal{D}_s$
    end for
    $\theta \leftarrow \mathcal{A}(\theta, \mathcal{D}_s)$ ▷ Update the policy model by PPO algorithm
    for i in $0, \cdots, M-1$ **do**
        if $r_i = 1$ **then** ▷ Update pass rate using moving average
            $\rho_i = \alpha + (1-\alpha) * \rho_i$
        else
            $\rho_i = (1-\alpha) * \rho_i$
        end if
        if $\rho_i > \rho_t$ **then** ▷ Meet the update conditions, proceed to the next stage
            $\rho_i = 0$
            $c_i = min(c_i - 1, 0)$
        end if
    end for
end while